Comparing Evaluation Function Paradigms in Othello

Jefferson David Kingston

1. Introduction

Building a program that plays Othello well is, at its core, a problem of evaluation. Othello is a two-player, perfect information game played on an 8x8 board where the goal is to end the game with more discs of your color than your opponent. Because the game tree contains roughly 10^58 possible positions, no program can search all the way to the end of every game in reasonable time. Instead, agents search ahead a fixed number of moves using minimax with alpha-beta pruning and rely on an evaluation function to assess positions at the search boundary. The quality of this evaluation function is arguably the most important factor in agent strength. A program with shallow search but an excellent evaluator will generally outplay one with deeper search but a poor one.

The most straightforward approach to building an evaluation function is to encode strategic knowledge directly. In Othello, the important features are reasonably well understood: corners are extremely valuable because they can never be flipped, stable pieces provide lasting positional security, mobility restricts the opponent's options, and raw piece count matters mainly in the endgame. The challenge is not identifying these features but weighting them correctly, and this turns out to be harder than it sounds. Alliot and Durand (1995) showed that manually tuning evaluation parameters is surprisingly difficult. Classical hill-climbing methods failed to converge to a stable optimum, and hand-tuned weights used by experienced programmers were significantly suboptimal. Their solution was to use a genetic algorithm to evolve the weights through tournament play. The evolved program won between 68 and 77 percent of points against the original baseline, and the evolved weights differed substantially from human intuition: the penalty for occupying X-squares adjacent to empty corners was far larger than expected, and mobility was considerably more important than the original program assumed.

A different line of work avoids feature design entirely. Tesauro (1995) demonstrated with TD-Gammon that a neural network trained through self-play using the TD(lambda) algorithm

could reach near world-class strength in backgammon without any encoded human knowledge. In some respects, the learned evaluation function surpassed human expert understanding. Tesauro noted that Othello is a particularly suitable domain for this approach, since expert beliefs about Othello strategy have historically shifted considerably, suggesting that any evaluation function anchored to current human knowledge may be missing something.

What is less clear is how these approaches compare directly. Alliot and Durand evaluated hand-crafted against GA-evolved weights but did not include a neural network. Tesauro demonstrated self-play training but did not benchmark it against a GA-evolved alternative. This project attempts to close that gap by implementing all three paradigms within a unified framework and comparing them under controlled conditions. All three agents use the same minimax search engine and are evaluated in the same round-robin tournament. The GA component follows Alliot and Durand's methodology using barycentric crossover, Gaussian mutation, and simulated annealing acceptance. The neural network is trained through self-play using a TD(lambda) update adapted from Tesauro, implemented as a multilayer perceptron in PyTorch.

2. Methods

2.1 Game Engine and Search

The Othello game engine was implemented in Python using NumPy. The board is represented as an 8x8 integer array where 1 denotes a black disc, -1 denotes a white disc, and 0 denotes an empty square. A square is a legal move if it is empty and placing a disc there would flip at least one opponent disc in at least one of the eight directions. When a move is applied, all flipped pieces are updated on a copy of the board rather than in place, preserving the original state for search. The game ends when neither player has legal moves, which can occur before the board is full. If one player has no moves but the other does, that turn is skipped, a rule known as a forced pass.

All three agents share the same search algorithm: minimax with alpha-beta pruning at a configurable depth, set to depth 5 for tournament play. Minimax assumes both players play optimally, alternating between maximising and minimising the evaluation score. Alpha-beta pruning maintains bounds alpha and beta and abandons any branch where beta falls below

alpha, since that branch cannot affect the final decision. The evaluation function is passed as a parameter through a shared (board, player) to float interface, so swapping between the three agents requires no changes to the search logic.

2.2 Hand-Crafted Evaluation Function

The hand-crafted evaluation function serves as the baseline for comparison. It scores a position as a weighted sum of four features motivated by standard Othello strategy.

Corner control rewards owning the four corner squares, which can never be flipped once captured. It also penalises occupying X-squares (diagonally adjacent to empty corners) and C-squares (edge squares adjacent to empty corners), since doing so frequently gifts the corner to the opponent. This structure follows Alliot and Durand's (1995) positional weighting scheme directly.

Stability counts the net number of discs that cannot be flipped for the remainder of the game regardless of future play, approximated by checking whether each disc is protected along all four axes by reaching the board edge or a fully occupied line.

Mobility is the normalised difference in legal move counts between the current player and the opponent, divided by the total moves available to both players to prevent it from dominating the evaluation early in the game.

Piece count is the raw disc differential. It is given the lowest weight because accumulating pieces early is widely considered a strategic mistake in Othello; fewer pieces often means better mobility and positional control.

The final score is: 25 x corner + 10 x stability + 5 x mobility + 1 x piece count. These weights reflect standard domain knowledge and serve as the human baseline the GA attempts to improve upon.

2.3 Genetic Algorithm

The genetic algorithm evolves the four feature weights by treating the vector [w_corner, w_stability, w_mobility, w_piece] as a chromosome optimised through simulated natural selection, following the methodology of Alliot and Durand (1995).

The population consists of 50 individuals, each a vector of four real-valued weights sampled uniformly within predefined bounds: corner in [1, 200], stability in [0, 100], mobility in [0, 50], and piece count in [0, 20]. One individual is seeded with the hand-crafted baseline weights to ensure a known-good solution is present from the start.

Fitness is evaluated by playing each individual against the hand-crafted baseline from randomised starting positions. Each evaluation plays four games (two starting positions, once as each colour) at search depth 2, chosen for computational feasibility. Alliot and Durand similarly used shallow depths for fitness evaluation. The fitness score combines win rate with a disc differential term to break ties, and is smoothed by averaging with the previous generation's fitness to reduce noise from the small game sample.

Selection uses binary tournament selection. Crossover follows Alliot and Durand's barycentric operator: a mixing coefficient alpha is sampled from [-0.5, 1.5] and a crossover point k from [1, 3], blending the k-th gene of each parent to produce two children. The extended range of alpha beyond [0, 1] allows children to extrapolate beyond their parents, promoting exploration. Crossover is applied with probability 0.9. Mutation adds zero-centred Gaussian noise to each weight with probability 0.15, using per-feature standard deviations of 50, 20, 10, and 5 for corner, stability, mobility, and piece count respectively.

After producing children, a simulated annealing acceptance criterion determines whether each child replaces its parent. Better children are always accepted; worse children are accepted with a probability that decays linearly from 0.5 at generation 1 to 0.01 at generation 50. Elitism carries the best individual forward unchanged each generation. The GA runs for 50 generations.

2.4 Neural Network

Rather than computing a weighted sum of predefined features, the neural network learns to evaluate board positions directly from raw input. The board is flattened to a 64-element vector encoded relative to the current player: the current player's discs as +1, the opponent's as -1, and empty squares as 0. This player-relative encoding means the network learns a single set of weights that generalises to both colours.

The architecture is a three-layer MLP: 64 inputs, a hidden layer of 128 units with ReLU activation, a second hidden layer of 64 units with ReLU activation, and a single output unit

with tanh activation mapping evaluations to (-1, 1). ReLU activations were chosen over the sigmoid units in the original TD-Gammon to avoid vanishing gradients and improve training speed.

Training follows Tesauro's (1995) self-play paradigm using a TD(lambda) update with lambda set to 0.7. Two copies of the network play against each other for 500 games at search depth 3. After each game, the network is updated so that earlier position evaluations align with later ones and the final outcome, with earlier positions receiving less weight through the lambda decay. The Adam optimiser is used with a learning rate of 0.001, minimising mean squared error between predicted evaluations and TD targets. Checkpoints are saved every 100 games.

2.5 Tournament Evaluation

The three agents are compared in a round-robin tournament where every ordered pair plays 20 games from randomised starting positions. Each matchup is played in both directions, with each agent taking both colours against every opponent, controlling for any first-mover advantage. Starting positions are generated by playing 8 to 18 random legal moves from the standard opening, ensuring deterministic agents face varied positions rather than replaying the same game. Results are reported as win rate (draws count as half a win), total points, and average disc differential. All agents use search depth 5 for tournament play.

3. Results

3.1 Genetic Algorithm Training

The genetic algorithm ran for 50 generations with a population of 50 individuals. Figure 1 shows the progression of mean and best fitness across generations.

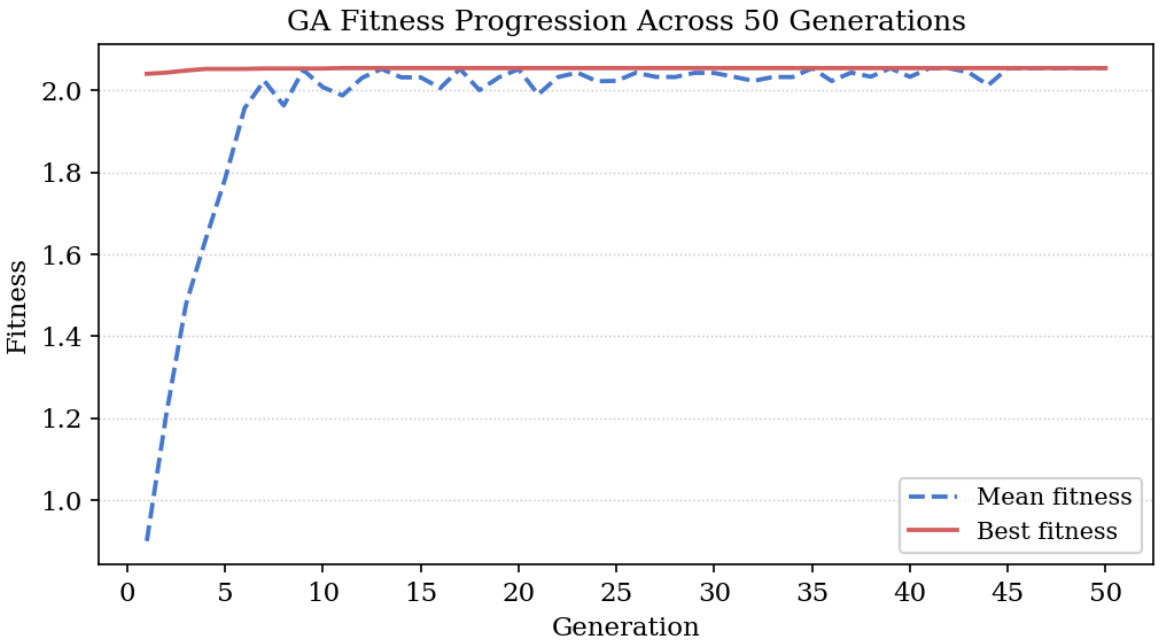

Figure 1: GA fitness progression across 50 generations. Mean population fitness converges rapidly toward the best individual fitness, with full population convergence achieved by generation 47.

Mean population fitness rose rapidly in the first ten generations, from 0.90 at generation 1 to 2.01 by generation 10, reflecting strong early selective pressure as the population moved away from random initialisation. The best individual fitness reached 2.055 by generation 11 and remained unchanged through generation 50, indicating the population converged on a stable solution early in the run. By generation 47, mean and best fitness were identical at 2.055, meaning the entire population had converged to the same weight configuration.

The final evolved weight vector was [163.72, 22.39, 38.44, 12.27].

Table 1: Comparison of hand-crafted and GA-evolved evaluation weights.

Feature	Hand-Crafted	GA-Evolved
Corner	25	163.72
Stability	10	22.39
Mobility	5	38.44
Piece count	1	12.27

The corner weight increased from 25.0 to 163.72 (a 555% increase), and the mobility weight increased from 5.0 to 38.44 (a 669% increase), both far exceeding the hand-crafted values. This pattern is consistent with Alliot and Durand's (1995) finding that human designers systematically undervalue these two features. The stability weight increased moderately from 10.0 to 22.39, while the piece count weight rose substantially from 1.0 to 12.27, suggesting that raw disc count carries more strategic weight at the search depths used in fitness evaluation than human intuition assumes.

3.2 Neural Network Training

The neural network was trained for 500 self-play games at search depth 3. Table 2 shows win rates and average TD loss at regular intervals.

Table 2: Neural network self-play training statistics across 500 games.

Game	**Black Win %**	**White Win %**	**Draw %**	**Avg TD Loss**
50	66.0%	30.0%	4.0%	0.0078
100	69.0%	27.0%	4.0%	0.0064
150	60.7%	35.3%	4.0%	0.0071
200	53.5%	43.0%	3.5%	0.0066

250	49.2%	47.6%	3.2%	0.0064
300	48.0%	48.7%	3.3%	0.0060
350	47.7%	48.9%	3.4%	0.0061
400	48.8%	47.8%	3.5%	0.0066
450	48.9%	47.3%	3.8%	0.0056
500	50.6%	45.6%	3.8%	0.0059

The most notable trend is the convergence of black and white win rates over time. Early in training, black won approximately 66 to 69 percent of games, reflecting the first-mover advantage that a randomly initialised network cannot overcome. By game 300, win rates had converged to roughly 48 percent for each side and remained stable through game 500. This suggests the network developed genuine positional understanding rather than simply exploiting the opening move advantage. The average TD loss decreased from 0.0078 at game 50 to 0.0059 at game 500, indicating that position evaluations became progressively more consistent with game outcomes. Over all 500 games, black won 253 (50.6%), white won 228 (45.6%), and 19 ended in draws (3.8%).

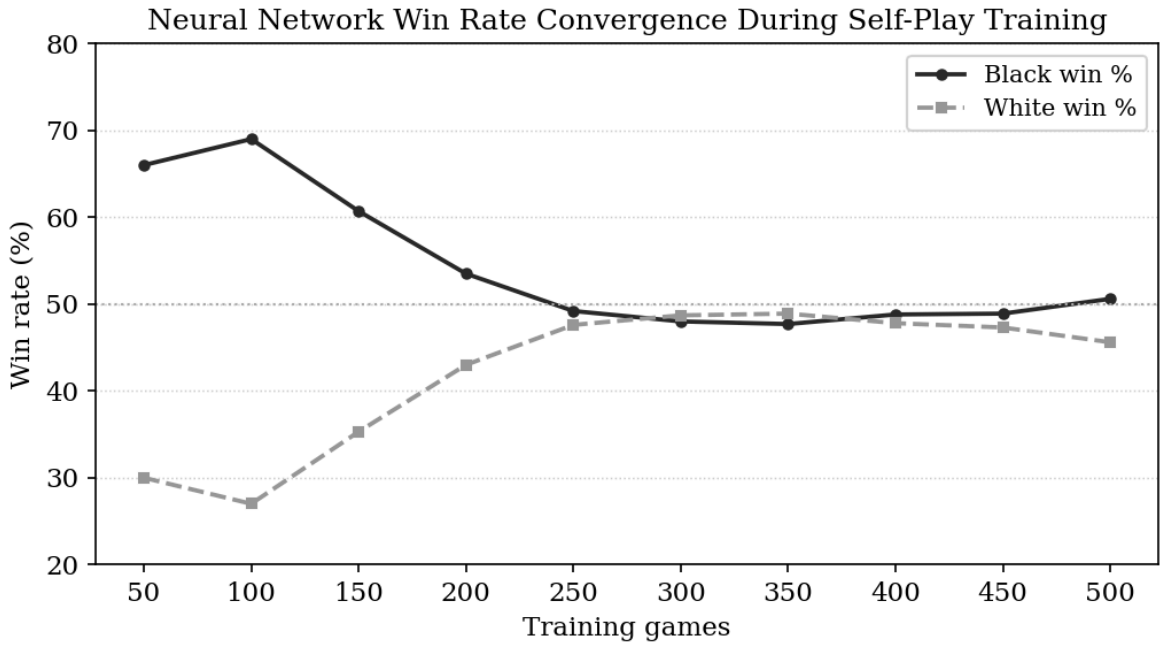

Figure 2: Neural network self-play win rates across 500 training games. Black and white win rates converge from an initial first-mover imbalance to near parity by game 300, indicating the development of genuine positional understanding.

3.3 Tournament Results

The three agents competed in a round-robin tournament with 20 games per ordered matchup at search depth 5. Each agent played every opponent as both colours, giving 40 games per head-to-head pairing and 80 total games per agent. Table 3 shows the final standings.

Table 3: Final tournament standings.

Agent	**W**	**L**	**D**	**Points**	**Win%**	**Avg Disc Diff**
Neural-Net	45	32	3	46.5	58.1%	-3.12
GA-Evolved	40	39	1	40.5	50.6%	+3.59
Hand-Crafted	31	45	4	33.0	41.2%	-0.46

The neural network won the tournament with a 58.1% win rate, finishing ahead of both heuristic agents. This result was unexpected given that the network trained for only 500 self-play games with no encoded domain knowledge. GA-Evolved finished second at 50.6%, just above the baseline of equal competition, while Hand-Crafted finished last at 41.2%. The disc differential results add nuance: GA-Evolved had the best average disc differential (+3.59), meaning it won more decisively when it did win, while the neural network's negative disc differential (-3.12) suggests it won by narrower margins but more consistently.

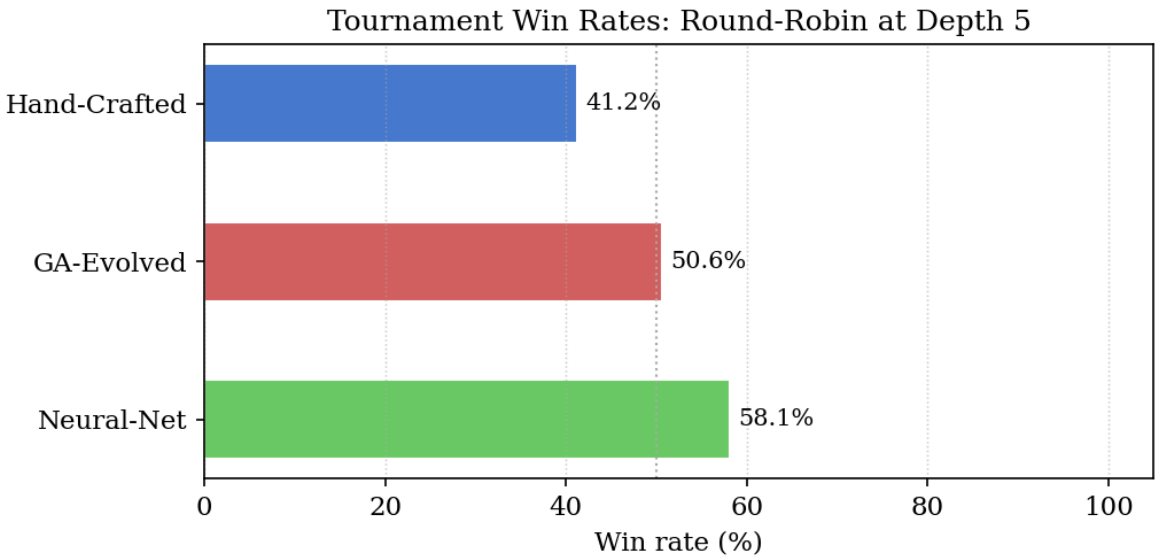

Figure 3: Tournament win rates for all three agents across 80 games each at search depth 5.

The matchup between Hand-Crafted and GA-Evolved produced results consistent with Alliot and Durand's (1995) findings. Combining both directions, GA-Evolved won 22 games and Hand-Crafted won 17, with 1 draw, giving GA-Evolved a 56.25% win rate in this head-to-head pairing. The evolved weights, particularly the dramatically higher corner and mobility values, translated into a meaningful competitive advantage over the hand-designed baseline.

The neural network's dominant performance against GA-Evolved as White in matchup 4 winning 15 of 20 games was the decisive factor in its overall victory. This result suggests the network learned an evaluation strategy that specifically counters the GA's aggressive corner-focused play. However, when the roles were reversed in matchup 6, GA-Evolved won 13 of 20 games as White against the neural network playing Black, indicating the network's advantage is not symmetric. The neural network performed strongly against Hand-Crafted in both directions, winning 23 of 40 games in that pairing, further confirming that 500 self-play games was sufficient to develop competitive strategic knowledge despite the absence of any encoded domain expertise.

4. Conclusions

This project compared three approaches to building evaluation functions for an Othello-playing agent within a unified framework: a hand-crafted heuristic, a GA-evolved heuristic, and a self-play trained neural network. All three agents used the same minimax search engine and were evaluated under identical tournament conditions, ensuring that performance differences reflected evaluation quality rather than implementation differences.

The most surprising result was that the neural network won the tournament outright with a 58.1% win rate, finishing ahead of both heuristic agents despite training on only 500 self-play games with no encoded domain knowledge. Its decisive performance against GA-Evolved as White, winning 15 of 20 games in that matchup, was the decisive factor in its overall victory and suggests the network learned an evaluation strategy that specifically counters corner-heavy play. The win rate convergence during training, from roughly 69 percent black wins early on to near 50-50 by game 300, confirmed that the network developed genuine positional understanding rather than simply exploiting the first-mover advantage.

The GA-evolved heuristic finished second at 50.6%, outperforming the hand-crafted baseline at 41.2% in overall play. The evolved weights showed the same pattern observed by Alliot and Durand (1995): corner control and mobility were substantially underweighted by human intuition, with the GA independently arriving at values far exceeding those a human designer would assign. The corner weight increased by 555% and the mobility weight by 669% relative to the hand-crafted baseline. That two independent evolutionary runs on different codebases converged on similar conclusions strengthens the case that these are genuine strategic insights rather than artifacts of a particular implementation.

There are several limitations worth acknowledging. The GA fitness evaluation used search depth 2 and only four games per individual per generation, both reduced for computational feasibility. The evolved weights may be better suited to shallow play than the depth 5 used in the tournament, and a larger evaluation budget would likely produce more reliable results. On the neural network side, the asymmetric performance across matchups, performing far better as White than as Black against the GA, points to inconsistencies in its strategic knowledge that more training would likely resolve. Tesauro's original TD-Gammon trained on hundreds of thousands of games and the results should be interpreted with that constraint in mind.

One of the more useful design decisions was the shared (board, player) to float interface across all three evaluation functions. This ensured the minimax engine, fitness evaluator, and tournament harness could all be written once and reused without modification, making it straightforward to attribute performance differences to evaluation quality rather than any other factor.

Future work could extend this comparison in several directions. A more computationally intensive GA run with deeper fitness evaluation would produce more reliable evolved weights. Training the neural network for more games would give it more opportunity to develop consistent strategic depth across both colours. It would also be worth examining whether the neural network independently learned to prioritise corner control and mobility as the GA did, or whether it discovered fundamentally different evaluation criteria that achieve competitive play through a different mechanism entirely. The asymmetry in its matchup results against the GA suggests the latter may be the case, which would make for a compelling follow-up investigation.

5. Works Cited

Alliot, J. M., & Durand, N. (1995, September). A genetic algorithm to improve an othello program. In European Conference on Artificial Evolution (pp. 305-319). Berlin, Heidelberg: Springer Berlin Heidelberg.

Tesauro, G. (1995). Temporal difference learning and TD-Gammon. Communications of the ACM, 38(3), 58-68.